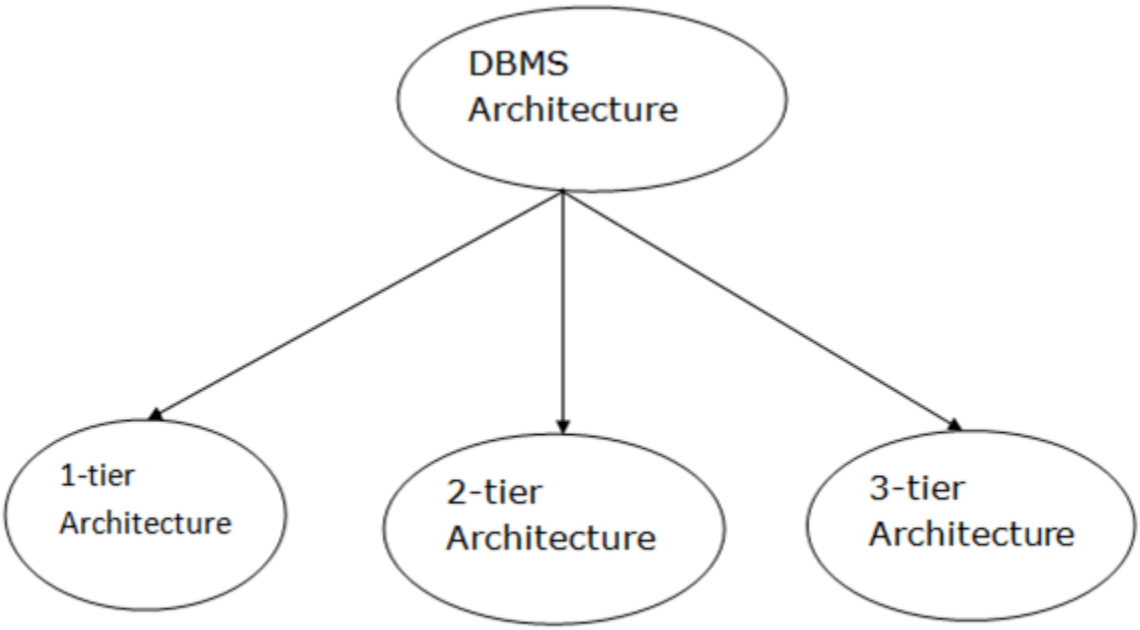
Unit-I

DBMS Architecture

- The DBMS design depends upon its architecture. The basic client/server architecture is used to deal with a large number of PCs, web servers, database servers and other components that are connected with networks.
- The client/server architecture consists of many PCs and a workstation which are connected via the network.
- DBMS architecture depends upon how users are connected to the database to get their request done.

Types of DBMS Architecture



Database architecture can be seen as a single tier or multi-tier. But logically, database architecture is of two types like: **2-tier architecture** and **3-tier architecture**.

1-Tier Architecture

- In this architecture, the database is directly available to the user. It means the user can directly sit on the DBMS and uses it.
- Any changes done here will directly be done on the database itself. It doesn't provide a handy tool for end users.
- The 1-Tier architecture is used for development of the local application, where programmers can directly communicate with the database for the quick response.

2-Tier Architecture

- The 2-Tier architecture is same as basic client-server. In the two-tier architecture, applications on the client end can directly communicate with the database at the server side. For this interaction, API's like: **ODBC**, **JDBC** are used.
- The user interfaces and application programs are run on the client-side.
- The server side is responsible to provide the functionalities like: query processing and transaction management.
- To communicate with the DBMS, client-side application establishes a connection with the server side.

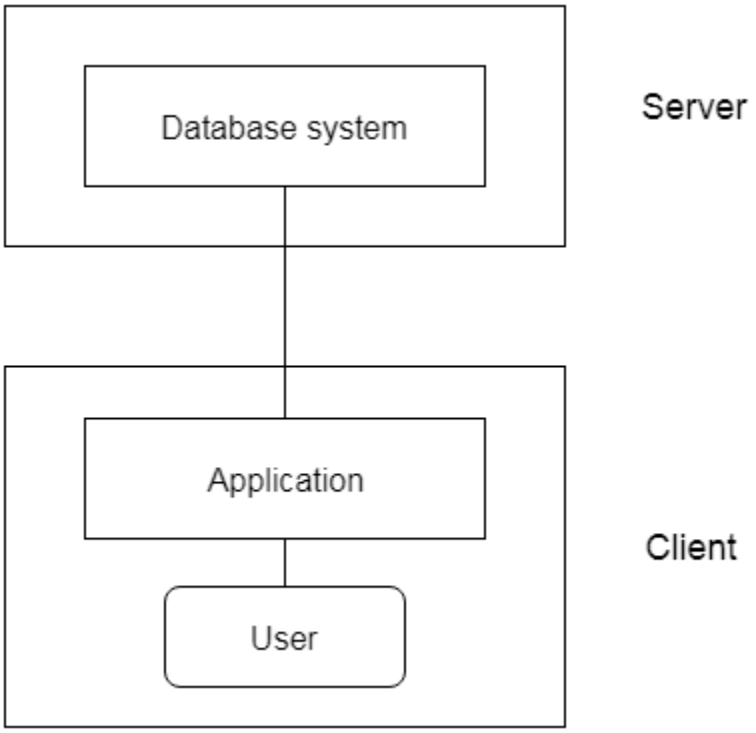


Fig: 2-tier Architecture

3-Tier Architecture

- The 3-Tier architecture contains another layer between the client and server. In this architecture, client can't directly communicate with the server.
- The application on the client-end interacts with an application server which further communicates with the database system.
- End user has no idea about the existence of the database beyond the application server. The database also has no idea about any other user beyond the application.
- The 3-Tier architecture is used in case of large web application.

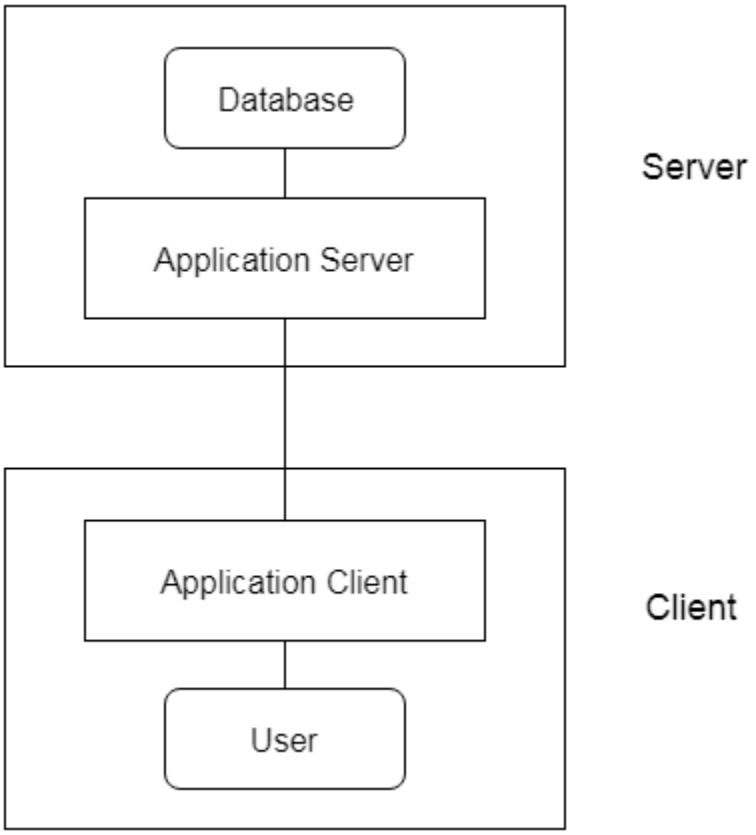


Fig: 3-tier Architecture

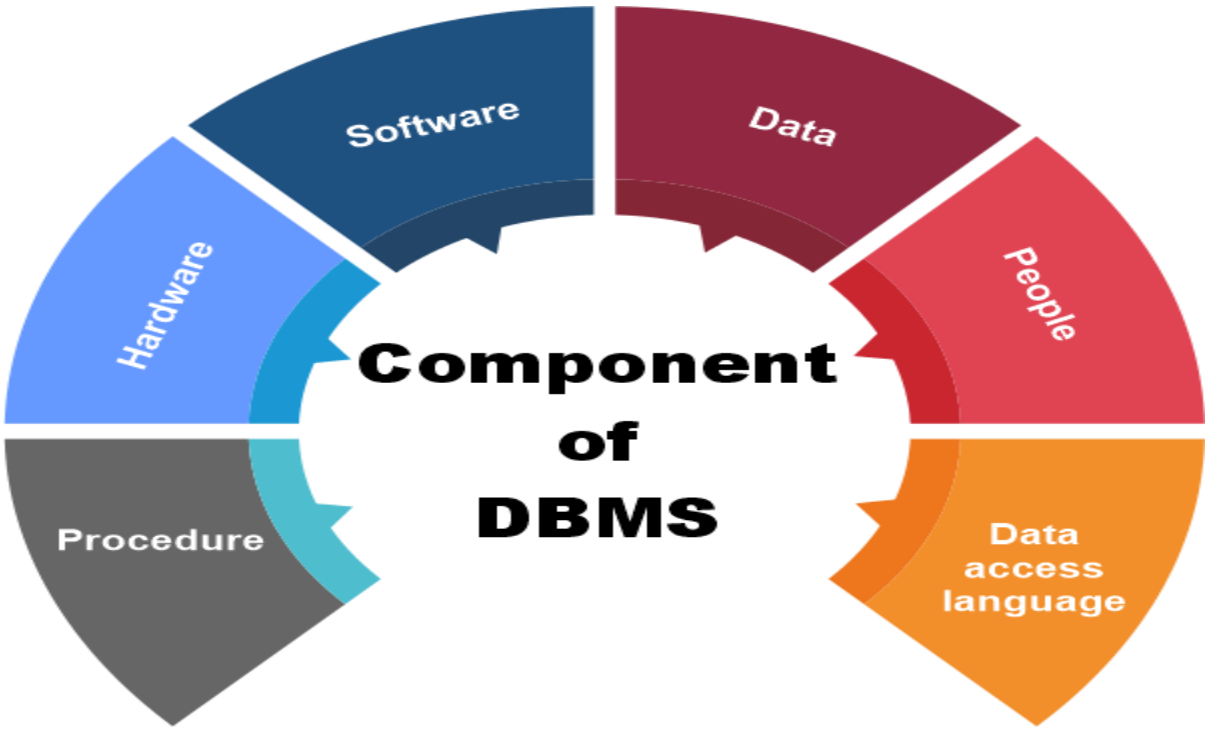
Components of DBMS

DBMS stands for DataBase Management System. DBMS is a type of software by which we can save and retrieve the user's data with the security process. DBMS can manipulate the database with the help of a group of programs. The DBMS can accept the request from the operating system to supply the data. The DBMS also can accept the request to retrieve a large amount of data through the user and third-party software.

DBMS also give permission to the user to use the data according to their needs. The word "DBMS" contains information regarding the database program and the users. It also provides an interface between the user and the software. In this topic, we are going to discuss the various types of DBMS.

Components of DBMS

There are many components available in the DBMS. Each component has a significant task in the DBMS. A database environment is a collection of components that regulates the use of data, management, and a group of data. These components consist of people, the technique of Handel the database, data, hardware, software, etc. there are several components available for the DBMS. We are going to explain five main topics of the database below.



1. Hardware

- Here the hardware means the physical part of the DBMS. Here the hardware includes output devices like a printer, monitor, etc., and storage devices like a hard disk.
- In DBMS, information hardware is the most important visible part. The equipment which is used for the visibility of the data is the printer, computer, scanner, etc. This equipment is used to capture the data and present the output to the user.
- With the help of hardware, the DBMS can access and update the database.
- The server can store a large amount of data, which can be shared with the help of the user's own system.
- The database can be run in any system that ranges from microcomputers to mainframe computers. And this database also provides an interface between the real worlds to the database.
- When we try to run any database software like MySQL, we can type any commands with the help of our keyboards, and RAM, ROM, and processor are part of our computer system.

2. Software

- Software is the main component of the DBMS.
- Software is defined as the collection of programs that are used to instruct the computer about its work. The software consists of a set of procedures, programs, and routines associated with the computer system's operation and performance. Also, we can say that computer software is a set of instructions that is used to instruct the computer hardware for the operation of the computers.
- The software includes so many software like network software and operating software. The database software is used to access the database, and the database application performs the task.
- This software has the ability to understand the database accessing language and then convert these languages to real database commands and then execute the database.
- This is the main component as the total database operation works on a software or application. We can also be called as database software the wrapper of the whole physical database, which provides an easy interface for the user to store, update and delete the data from the database.

- Some examples of DBMS software include MySQL, Oracle, SQL Server, dBase, FileMaker, Clipper, Foxpro, Microsoft Access, etc.

3. Data

- The term data means the collection of any raw fact stored in the database. Here the data are any type of raw material from which meaningful information is generated.
- The database can store any form of data, such as structural data, non-structural data, and logical data.
- The structured data are highly specific in the database and have a structured format. But in the case of non-structural data, it is a collection of different types of data, and these data are stored in their native format.
- We also call the database the structure of the DBMS. With the help of the database, we can create and construct the DBMS. After the creation of the database, we can create, access, and update that database.
- The main reason behind discovering the database is to create and manage the data within the database.
- Data is the most important part of the DBMS. Here the database contains the actual data and metadata. Here metadata means data about data.
- For example, when the user stores the data in a database, some data, such as the size of the data, the name of the data, and some data related to the user, are stored within the database. These data are called metadata.

4. Procedures

- The procedure is a type of general instruction or guidelines for the use of DBMS. This instruction includes how to set up the database, how to install the database, how to log in and log out of the database, how to manage the database, how to take a backup of the database, and how to generate the report of the database.
- In DBMS, with the help of procedure, we can validate the data, control the access and reduce the traffic between the server and the clients. The DBMS can offer better performance to extensive or complex business logic when the user follows all the procedures correctly.
- The main purpose of the procedure is to guide the user during the management and operation of the database.

- The procedure of the databases is so similar to the function of the database. The major difference between the database procedure and database function is that the database function acts the same as the SQL statement. In contrast, the database procedure is invoked using the CALL statement of the DBMS.
- Database procedures can be created in two ways in enterprise architecture. These two ways are as below.
- The individual object or the default object.
- The operations in a container.

```
1. CREATE [OR REPLACE] PROCEDURE procedure_name (<Argument> {IN, OUT, IN OUT}

2. <Datatype>,...)
3. IS
4.   Declaration section<variable, constant> ;
5. BEGIN
6.   Execution section
7. EXCEPTION
8.   Exception section
9. END
```

5. Database Access Language

- Database Access Language is a simple language that allows users to write commands to perform the desired operations on the data that is stored in the database.
- Database Access Language is a language used to write commands to access, upsert, and delete data stored in a database.
- Users can write commands or query the database using Database Access Language before submitting them to the database for execution.
- Through utilizing the language, users can create new databases and tables, insert data and delete data.
- Examples of database languages are SQL (structured query language), My Access, Oracle, etc. A database language is comprised of two languages.

1. Data Definition Language(DDL): It is used to construct a database. DDL implements database schema at the physical, logical, and external levels.

The following commands serve as the base for all DDL commands:

- ALTER<object>
- COMMENT
- CREATE<object>
- DESCRIBE<object>
- DROP<object>
- SHOW<object>
- USE<object>

2. Data Manipulation Language(DML): It is used to access a database. The DML provides the statements to retrieve, modify, insert and delete the data from the database.

The following commands serve as the base for all DML commands:

- INSERT
- UPDATE
- DELETE
- LOCK
- CALL
- EXPLAIN PLAN

6. People

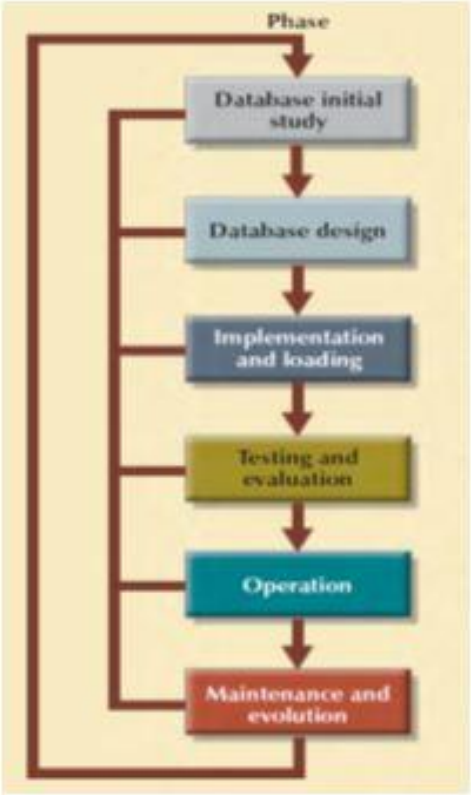
- The people who control and manage the databases and perform different types of operations on the database in the DBMS.
- The people include database administrator, software developer, and End-user.
- Database administrator-database administrator is the one who manages the complete database management system. DBA takes care of the security of the DBMS, its availability, managing the license keys, managing user accounts and access, etc.
- Software developer- theThis user group is involved in developing and designing the parts of DBMS. They can handle massive quantities of data, modify and edit databases, design and develop new databases, and troubleshoot database issues.

- End user - These days, all modern web or mobile applications store user data. How do you think they do it? Yes, applications are programmed in such a way that they collect user data and store the data on a DBMS system running on their server. End users are the ones who store, retrieve, update and delete data.
- The users of the database can be classified into different groups.
    i. Native Users
    ii. Online Users
    iii. Sophisticated Users
    iv. Specialized Users
    v. Application Users
    vi. DBA - Database Administrator

Database Development Life Cycle

The Database Life Cycle (DBLC)

The Database Life Cycle (DBLC) contains six phases, as shown in the following Figure: database initial study, database design, implementation and loading, testing and evaluation, operation, and maintenance and evolution.



The Database Life Cycle (DBLC)

1. The Database Initial Study:

In the Database initial study, the designer must examine the current system's operation within the company and determine how and why the current system fails. The overall purpose of the database initial study is to:

- Analyze the company situation.
- Define problems and constraints.
- Define objectives.
- Define scope and boundaries.

a. Analyze the Company Situation:

The company situation describes the general conditions in which a company operates, its organizational structure, and its mission. To analyze the company situation, the database designer must discover what the company's operational components are, how they function, and how they interact.

b. Define Problems and Constraints:

The designer has both formal and informal sources of information. The process of defining problems might initially appear to be unstructured. Company end users are often unable to describe precisely the larger scope of company operations or to identify the real problems encountered during company operations.

c. Define Objectives:

A proposed database system must be designed to help solve at least the major problems identified during the problem discovery process. In any case, the database designer must begin to address the following questions:

- What is the proposed system's initial objective?
- Will the system interface with other existing or future systems in the company?
- Will the system share the data with other systems or users?

d. Define Scope and Boundaries:

The designer must recognize the existence of two sets of limits: scope and boundaries. The system's scope defines the extent of the design according to operational requirements. Will the database design encompass the entire organization, one or more departments within the organization, or one or more functions of a single department? Knowing the scope helps in defining the required data structures, the type and number of entities, the physical size of the database, and so on.

The proposed system is also subject to limits known as boundaries, which are external to the system. Boundaries are also imposed by existing hardware and software.

2. Database Design:

The second phase focuses on the design of the database model that will support company operations and objectives. This is arguably the most critical DBLC phase: making sure that the final product meets user and system requirements. As you examine the procedures required to complete the design phase in the DBLC, remember these points:

- The process of database design is loosely related to the analysis and design of a larger system. The data component is only one element of a larger information system.

- The systems analysts or systems programmers are in charge of designing the other system components. Their activities create the procedures that will help transform the data within the database into useful information.

3. Implementation and Loading:

The output of the database design phase is a series of instructions detailing the creation of tables, attributes, domains, views, indexes, security constraints, and storage and performance guidelines. In this phase, you actually implement all these design specifications.

a. Install the DBMS:

This step is required only when a new dedicated instance of the DBMS is necessary for the system. The DBMS may be installed on a new server or it may be installed on existing servers. One current trend is called virtualization. Virtualization is a technique that creates logical representations of computing resources that are independent of the underlying physical computing resources.

b. Create the Database(s):

In most modern relational DBMSs a new database implementation requires the creation of special storage-related constructs to house the end-user tables. The

constructs usually include the storage group (or file groups), the table spaces, and the tables.

c. Load or Convert the Data:

After the database has been created, the data must be loaded into the database tables. Typically, the data will have to be migrated from the prior version of the system. Often, data to be included in the system must be aggregated from multiple sources. Data may have to be imported from other relational databases, non relational databases, flat files, legacy systems, or even manual paper-and-pencil systems

4. Testing and Evaluation:

In the design phase, decisions were made to ensure integrity, security, performance, and recoverability of the database. During implementation and loading, these plans were put into place. In testing and evaluation, the DBA tests and fine-tunes the database to ensure that it performs as expected. This phase occurs in conjunction with applications programming.

a. Test the Database:

During this step, the DBA tests the database to ensure that it maintains the integrity and security of the data. Data integrity is enforced by the DBMS through the proper use of primary and foreign key rules. In database testing you must check Physical security allows, Password security, Access rights, Data encryption etc.

b. Fine-Tune the Database:

Although database performance can be difficult to evaluate because there are no standards for database performance measures, it is typically one of the most important factors in database implementation. Different systems will place different performance requirements on the database. Many factors can impact the database's performance on various tasks. Environmental factors, such as the hardware and software environment in which the database exists, can have a significant impact on database performance.

c. Evaluate the Database and Its Application Programs:

As the database and application programs are created and tested, the system must also be evaluated from a more holistic approach. Testing and evaluation of the individual components should culminate in a variety of broader system tests to ensure that all of the components interact properly to meet the needs of the users. To ensure that the data contained in the database are protected against loss, backup and recovery plans are tested.

5. Operation

Once the database has passed the evaluation stage, it is considered to be operational. At that point, the database, its management, its users, and its application programs constitute a complete information system. The beginning of the operational phase invariably starts the process of system evolution.

6. Maintenance and Evolution

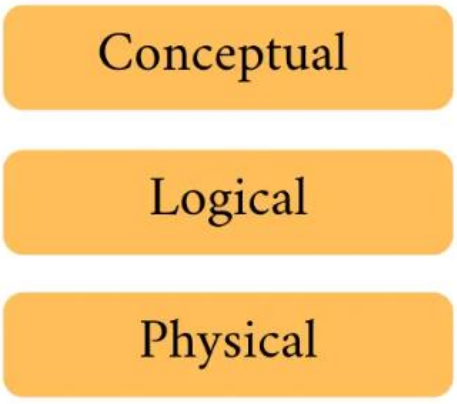
The database administrator must be prepared to perform routine maintenance activities within the database. Some of the required periodic maintenance activities include:

- Preventive maintenance (backup).
- Corrective maintenance (recovery).
- Adaptive maintenance (enhancing performance, adding entities and attributes, and so on).
- Assignment of access permissions and their maintenance for new and old users.

Conceptual data modeling

To minimize the risks, the designer will usually start off by data modeling.

There are three stages in data modeling: conceptual, logical, and physical. Each stage brings the database closer to reality.



The conceptual model sketches out the entities to be represented and determines what kinds of relationships exist between them. It deals with the scope of the database to be created and defines the general rules that need to be considered.

The logical model will take these entities a step further and work out the details of how their attributes and relationships. It defines the structure, but does not concern itself with the technical aspects of how the database will be constructed.

The physical model moves from abstraction to reality and considers the database management technology to be used, the design of the tables that will make up the actual database, and the keys that will represent the relationships between these tables.

What is the purpose of a conceptual data model?

The conceptual data model gives the designer the chance to gain an overview of the system to be designed without being concerned with the details of how it will be implemented. Conceptual data models can be very quick to create, but they can also rapidly highlight faulty assumptions and potential problems. The conceptual model is a simplified diagram of the final database, with the details deliberately ignored so that the big picture can be understood.

9 characteristics of a good conceptual data model

The ideal conceptual data model will do all of the following.

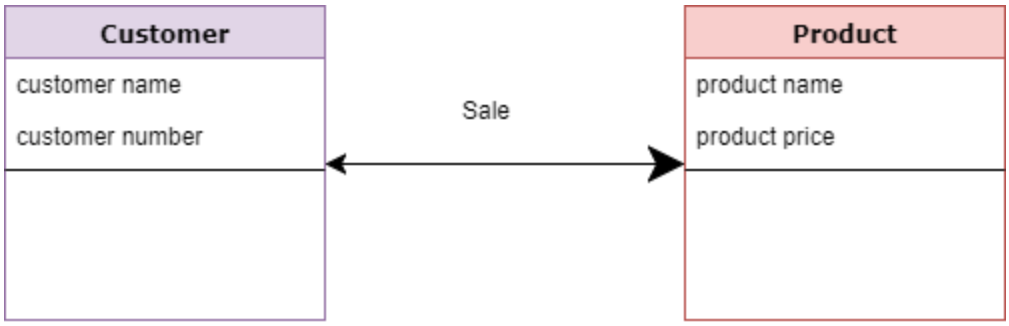
1. Provide a high-level overview of the system to be built.
2. Define the scope of the data to be represented.
3. Create a blueprint that can be referred to throughout the project.
4. Diagram entities and relationships rather than attributes.
5. Avoid dealing with technical considerations or terminology.
6. Prevent the model from already being tied to a particular database management system.
7. Be used to get feedback from non-technical stakeholders.
8. Focus on the business requirements the database needs to solve.
9. Provide a solid foundation for creating logical and physical models.

How to create a conceptual data model

Entity-relationship models are one of the most popular ways to create a quick and clear conceptual data model. An ER model consists of entities, attributes, and relationships.

Data model example:

- Customer and Product are two entities. Customer number and name are attributes of the Customer entity
- Product name and price are attributes of product entity
- Sale is the relationship between the customer and product



Conceptual Data Model

Characteristics of a conceptual data model

- Offers Organisation-wide coverage of the business concepts.
- This type of Data Models are designed and developed for a business audience.
- The conceptual model is developed independently of hardware specifications like data storage capacity, location or software specifications like DBMS vendor and technology. The focus is to represent data as a user will see it in the "real world."

Conceptual data models known as Domain models create a common vocabulary for all stakeholders by establishing basic concepts and scope.

1. **Entities**

The real-world elements in the system are defined first. Entities can be concepts, events, objects, persons, companies, or systems. If a thing can be identified as discrete, then it can be an entity.

2. **Attributes**

The characteristics that differentiate the entity from other entities are its attributes. These can be anything that defines the entity, such as name, category, ID, date of creation, or description. The types of attributes will depend on the type of system being created.

3. **Relationships**

Each entity in a database has some sort of relationship with the other entities. They may be related because they interact with each other, or they may be dependent on each other, or have a parent-child relationship, with inheritance of attributes. Whatever the relationship, it needs to be explained and represented during the process of data modeling.

4. **Normalization**

Database normalization is based on removing ambiguities from relationships and removing repetition or redundancy. This is carried out by reviewing the various primary, secondary, or foreign keys associated with each entity.

5. **Validation**

This phase is hopefully when it all comes together, with the rules, formats, requirements, and syntax being checked, along with the entities, relationships and keys. If changes are needed, the designer goes over the model again to refine it.

What is a primary key?

Primary keys are columns in a table that uniquely identify the information in rows or tuples. Each table has just one primary key that can have several attributes, and are used to identify information within the table. Without a primary key, finding information in the table would be extremely difficult, if not impossible.

When to use a primary key

Without primary keys, specifying individual records wouldn't be possible. We see examples of primary keys in real life, which are known in the software world as natural primary keys. Some examples of natural primary keys – things used to identify people or things specifically – are ID numbers, addresses, and vehicle identification numbers. **Each of these keys can identify only one item.**

What is a foreign key?

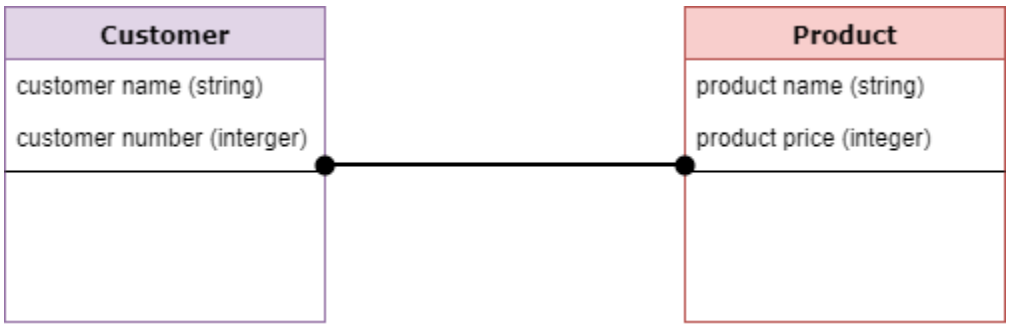
A foreign key is one or more columns in a table that references the primary key of another, creating a link between them. Foreign keys cannot exist without being linked to a primary key. Unlike primary keys, tables can have more than one foreign key.

When to use foreign keys: Foreign keys act like primary keys in that they are also used to identify specific entries in tuples, but foreign keys always reference another table. Tables with foreign keys are called 'child tables', because they always link back to a table with a primary KEY.

Features	**Primary keys**	**Foreign keys**
In tables	Part of a parent table	Part of a child table, always links back to a primary key in a parent table
Number	Only one per parent table	Tables can have multiple foreign keys
Values	Must have a value, and every item must be identified	Can have the value of NULL
Ease	Cannot be removed from the table	Can be removed from the table

Logical Data Model

The Logical Data Model is used to define the structure of data elements and to set relationships between them. The logical data model adds further information to the conceptual data model elements. The advantage of using a Logical data model is to provide a foundation to form the base for the Physical model. However, the modeling structure remains generic.



Logical Data Model

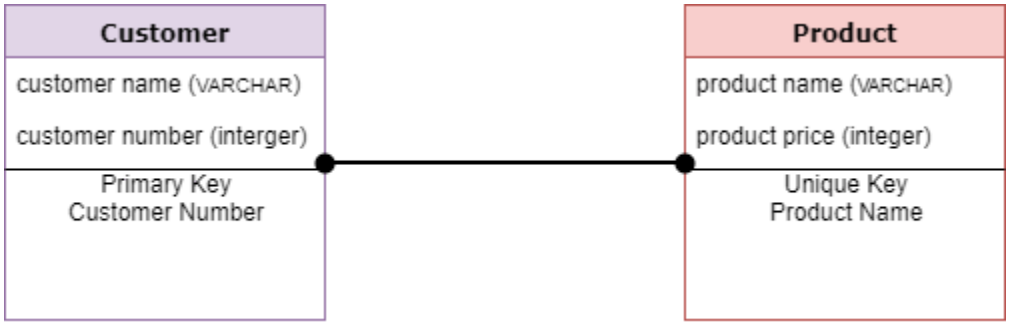
At this Data Modeling level, no primary or secondary key is defined. At this Data modeling level, you need to verify and adjust the connector details that were set earlier for relationships.

Characteristics of a Logical data model

- Describes data needs for a single project but could integrate with other logical data models based on the scope of the project.
- Designed and developed independently from the DBMS.
- Data attributes will have datatypes with exact precisions and length.
- Normalization processes to the model is applied typically till 3NF.

Physical Data Model

A **Physical Data Model** describes a database-specific implementation of the data model. It offers database abstraction and helps generate the schema. This is because of the richness of meta-data offered by a Physical Data Model. The physical data model also helps in visualizing database structure by replicating database column keys, constraints, indexes, triggers, and other RDBMS features.



Physical Data Model

Characteristics of a physical data model

- The physical data model describes data need for a single project or application though it maybe integrated with other physical data models based on project scope.
- Data Model contains relationships between tables that which addresses cardinality and nullability of the relationships.
- Developed for a specific version of a DBMS, location, data storage or technology to be used in the project.

- Columns should have exact datatypes, lengths assigned and default values.
- Primary and Foreign keys, views, indexes, access profiles, and authorizations, etc. are defined.

Advantages of Data Modeling

The advantages of going through the data modeling process all come down to communication:

- Short term communication among stakeholders to make decisions about what's important, what the business rules are, and how to implement them.
- Long term communication through database specifications that can be used to connect your data to other services through ETLs (Couchbase can help you reduce the number of ETLs, as there are a variety of built-in services to help address your expanding use cases – query, text search, caching, analytics, eventing, mobile sync).
- Communication to help your team more easily identify corrupt or incorrect data.

Disadvantages of Data Modeling

There are costs to data modeling.

- It can be a potentially long process. It can also be prone to waterfall mentality (e.g. a mistake found during the logical data modeling process could trigger a complete rework of the conceptual modeling process).
- A physical relational model can be rigid and difficult to change once a physical data model has been created (especially in production).
- A physical document model is easy to change at any time, but relies on the application layer to enforce constraints and data types.
- With Couchbase's document model, you can still use JOIN and ACID transactions when necessary, so the modeling process should be familiar to anyone who is used to relational modeling, but with added flexibility and data structures that line up exactly with application code objects/classes.

ER Model

The Entity Relational Model is a model for identifying entities to be represented in the database and representation of how those entities are related. The ER data model specifies enterprise schema that represents the overall logical structure of a database graphically.

The Entity Relationship Diagram explains the relationship among the entities present in the database. ER models are used to model real-world objects like a person, a car, or a company and the relation between these real-world objects. In short, the ER Diagram is the structural format of the database.

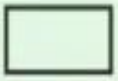
Why Use ER Diagrams In DBMS?

- ER diagrams are used to represent the E-R model in a database, which makes them easy to be converted into relations (tables).
- ER diagrams provide the purpose of real-world modeling of objects which makes them intently useful.
- ER diagrams require no technical knowledge and no hardware support.
- These diagrams are very easy to understand and easy to create even for a naive user.
- It gives a standard solution for visualizing the data logically.

Symbols Used in ER Model

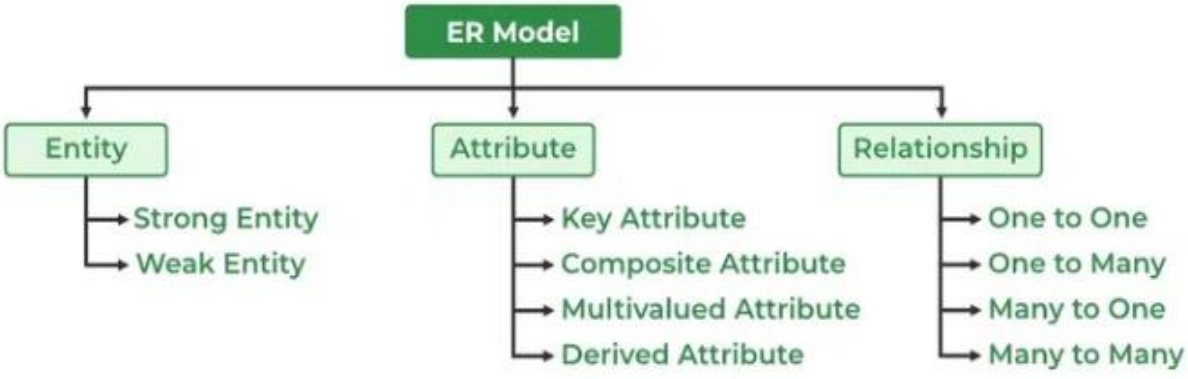
ER Model is used to model the logical view of the system from a data perspective which consists of these symbols:

- **Rectangles:** Rectangles represent Entities in the ER Model.
- **Ellipses:** Ellipses represent Attributes in the ER Model.
- **Diamond:** Diamonds represent Relationships among Entities.
- **Lines:** Lines represent attributes to entities and entity sets with other relationship types.
- **Double Ellipse:** Double Ellipses represent Multi-Valued Attributes.
- **Double Rectangle:** Double Rectangle represents a Weak Entity.

Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

Components of ER Diagram

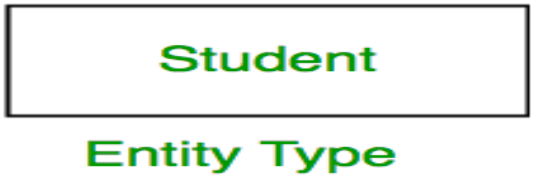
ER Model consists of Entities, Attributes, and Relationships among Entities in a Database System.



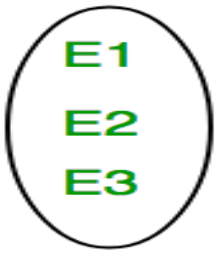
Entity

An Entity may be an object with a physical existence – a particular person, car, house, or employee – or it may be an object with a conceptual existence – a company, a job, or a university course.

Entity Set: An Entity is an object of Entity Type and a set of all entities is called an entity set. For Example, E1 is an entity having Entity Type Student and the set of all students is called Entity Set. In ER diagram, Entity Type is represented as:



Entity Type



Entity Set

1. Strong Entity

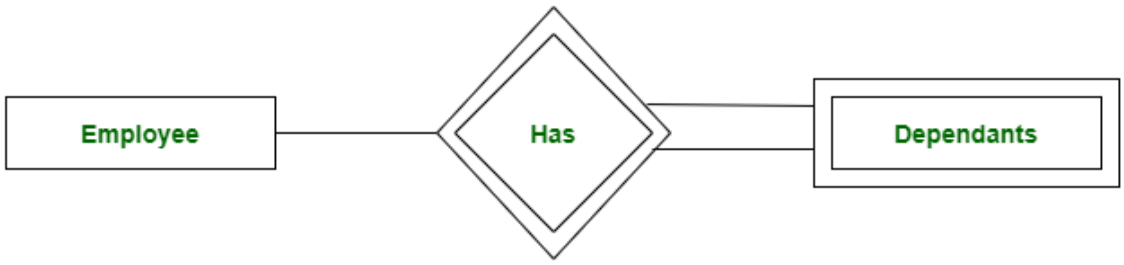
A Strong Entity is a type of entity that has a key Attribute. Strong Entity does not depend on other Entity in the Schema. It has a primary key, that helps in identifying it uniquely, and it is represented by a rectangle. These are called Strong Entity Types.

2. Weak Entity

An Entity type has a key attribute that uniquely identifies each entity in the entity set. But some entity type exists for which key attributes can't be defined. These are called Weak Entity types.

For Example, A company may store the information of dependents (Parents, Children, Spouse) of an Employee. But the dependents don't have existed without the employee. So Dependent will be a **Weak Entity Type** and Employee will be Identifying Entity type for Dependent, which means it is **Strong Entity Type**.

A weak entity type is represented by a Double Rectangle. The participation of weak entity types is always total. A double diamond represents the relationship between the weak entity type and its identifying strong entity type is called identifying relationship and it.



Strong Entity and Weak Entity

Attributes

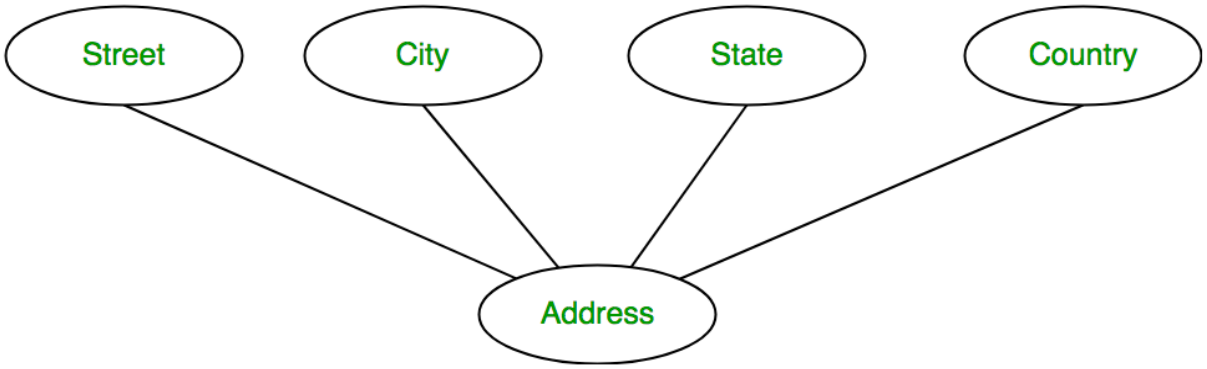
Attributes are the properties that define the entity type. For example, Roll_No, Name, DOB, Age, Address, and Mobile_No are the attributes that define entity type Student. In ER diagram, the attribute is represented by an oval.

1. Key Attribute

The attribute which **uniquely identifies each entity** in the entity set is called the key attribute. For example, Roll_No will be unique for each student. In ER diagram, the key attribute is represented by an oval with underlying lines.

2. Composite Attribute

An attribute **composed of many other attributes** is called a composite attribute. For example, the Address attribute of the student Entity type consists of Street, City, State, and Country. In ER diagram, the composite attribute is represented by an oval comprising of ovals.



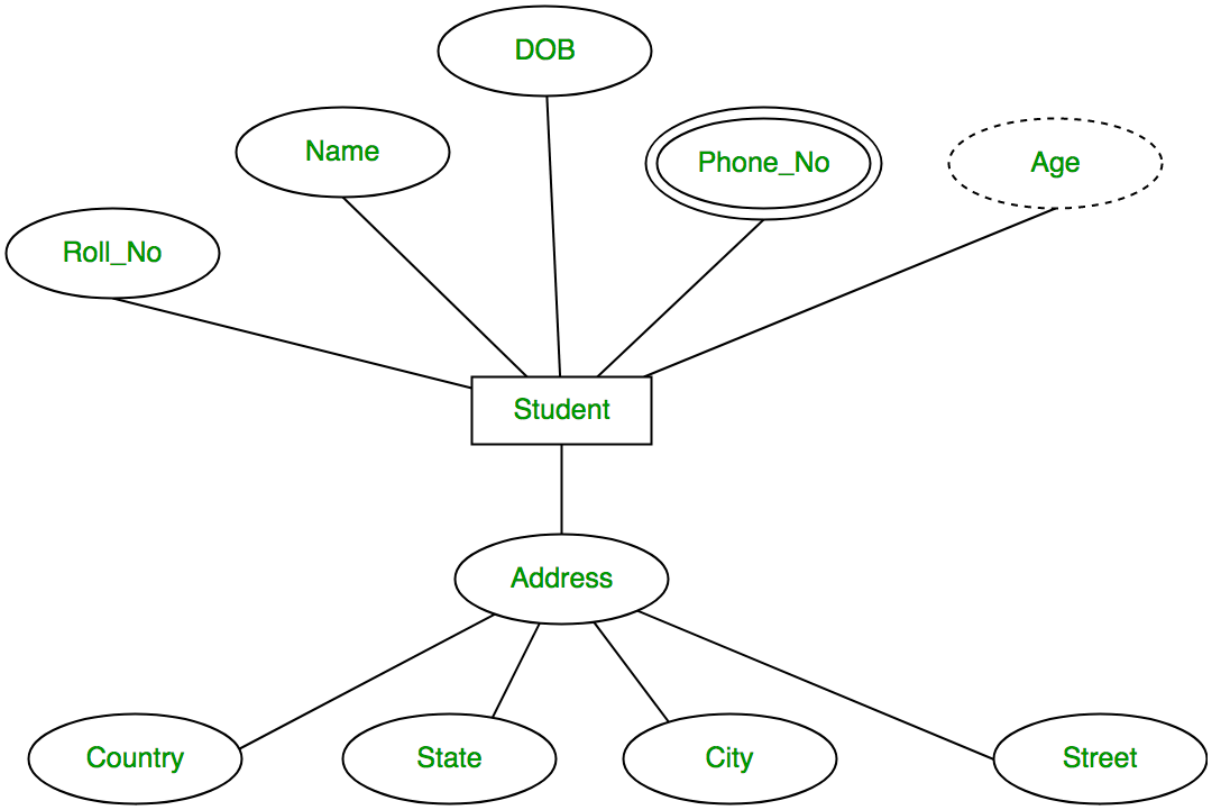
3. Multivalued Attribute

An attribute consisting of more than one value for a given entity. For example, Phone_No (can be more than one for a given student). In ER diagram, a multivalued attribute is represented by a double oval.

4. Derived Attribute

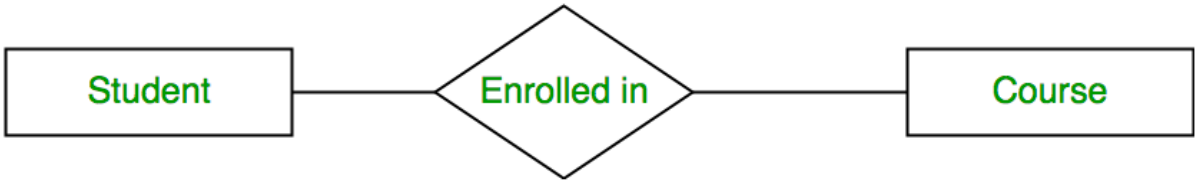
An attribute that can be derived from other attributes of the entity type is known as a derived attribute. e.g.; Age (can be derived from DOB). In ER diagram, the derived attribute is represented by a dashed oval.

The Complete Entity Type Student with its Attributes can be represented as:

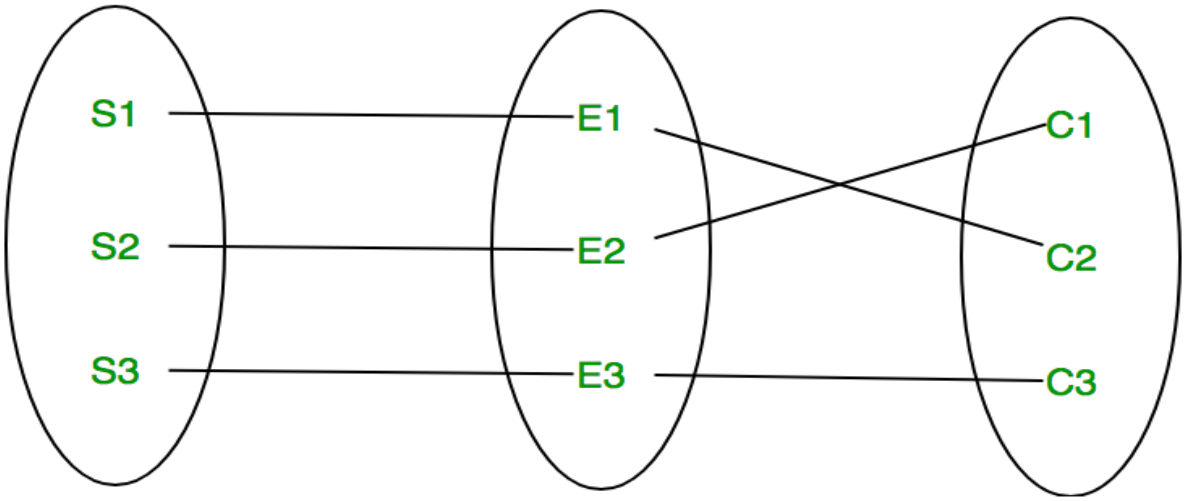


Relationship Type and Relationship Set

A Relationship Type represents the association between entity types. For example, 'Enrolled in' is a relationship type that exists between entity type Student and Course. In ER diagram, the relationship type is represented by a diamond and connecting the entities with lines.



A set of relationships of the same type is known as a relationship set. The following relationship set depicts S1 as enrolled in C2, S2 as enrolled in C1, and S3 as registered in C3.

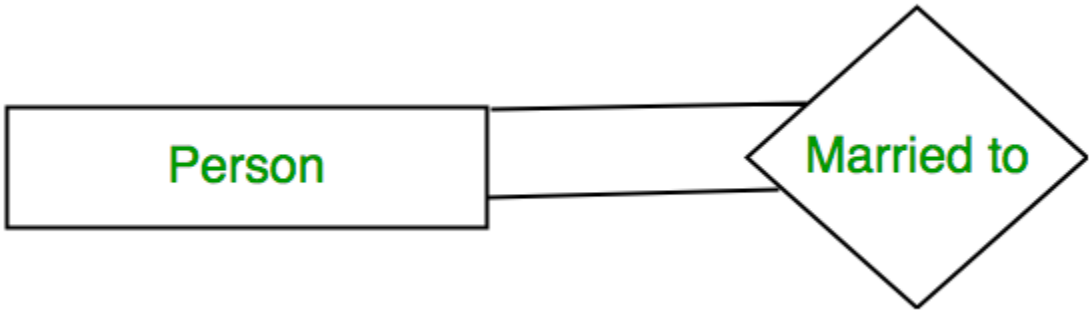


Relationship Set

Degree of a Relationship Set

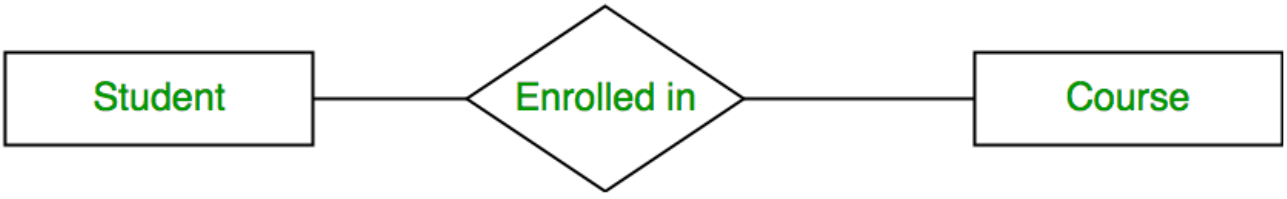
The number of different entity sets participating in a relationship set is called the degree of a relationship set.

1. Unary Relationship: When there is only ONE entity set participating in a relation, the relationship is called a unary relationship. For example, one person is married to only one person



Unary Relationship

2. Binary Relationship: When there are TWO entities set participating in a relationship, the relationship is called a binary relationship. For example, a Student is enrolled in a Course.



Binary Relationship

3. n-ary Relationship: When there are n entities set participating in a relation, the relationship is called an n-ary relationship.

Cardinality

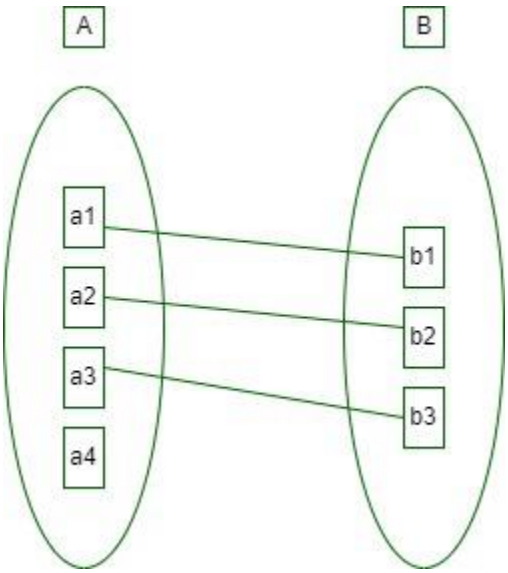
The number of times an entity of an entity set participates in a relationship set is known as cardinality. Cardinality can be of different types:

1. One-to-One: When each entity in each entity set can take part only once in the relationship, the cardinality is one-to-one. Let us assume that a male can marry one female and a female can marry one male. So the relationship will be one-to-one.

the total number of tables that can be used in this is 2.

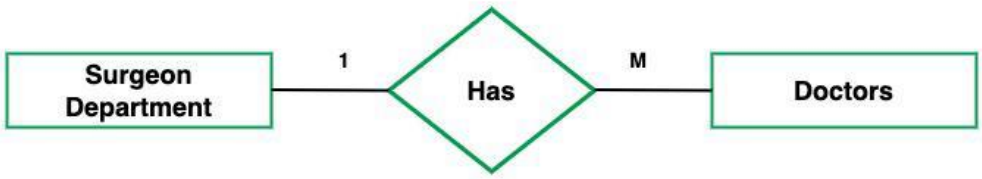
one to one cardinality

Using Sets, it can be represented as:



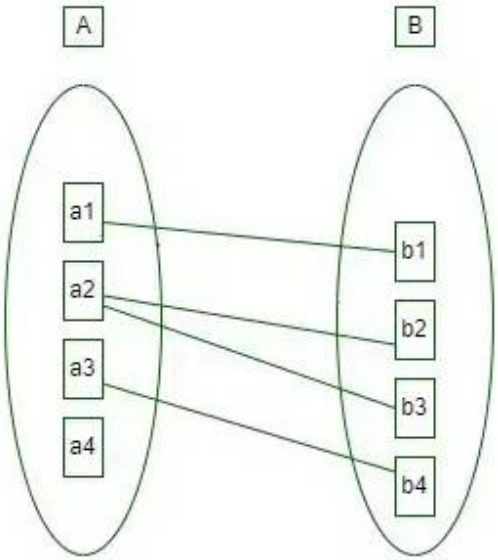
2. One-to-Many: In one-to-many mapping as well where each entity can be related to more than one relationship and the total number of tables that can be used in this is 2. Let us assume that one surgeon deparment can accomodate many doctors. So the Cardinality will be 1 to M. It means one deparment has many Doctors.

total number of tables that can used is 3.



one to many cardinality

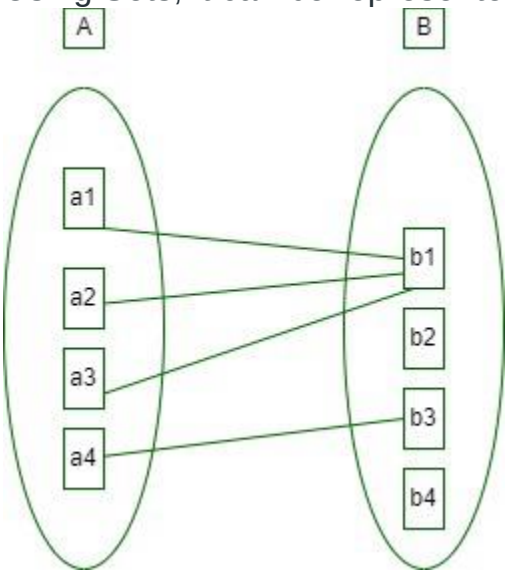
Using sets, one-to-many cardinality can be represented as:



3. Many-to-One: When entities in one entity set can take part only once in the relationship set and entities in other entity sets can take part more than once in the relationship set, cardinality is many to one. Let us assume that a student can take only one course but one course can be taken by many students. So the cardinality will be n to 1. It means that for one course there can be n students but for one student, there will be only one course.
The total number of tables that can be used in this is 3.

many to one cardinality

Using Sets, it can be represented as:

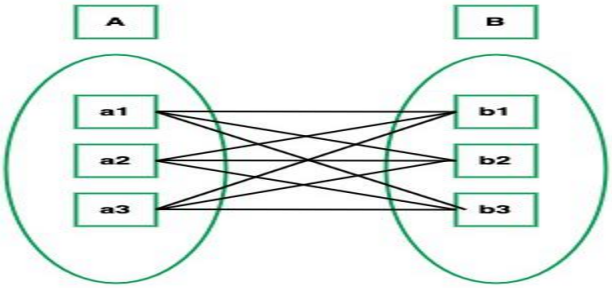


In this case, each student is taking only 1 course but 1 course has been taken by many students.

4. Many-to-Many: When entities in all entity sets can take part more than once in the relationship cardinality is many to many. Let us assume that a student can take more than one course and one course can be taken by many students. So the relationship will be many to many.
the total number of tables that can be used in this is 3.

many to many cardinality

Using Sets, it can be represented as:



In this example, student S1 is enrolled in C1 and C3 and Course C3 is enrolled by S1, S3, and S4. So it is many-to-many relationships.

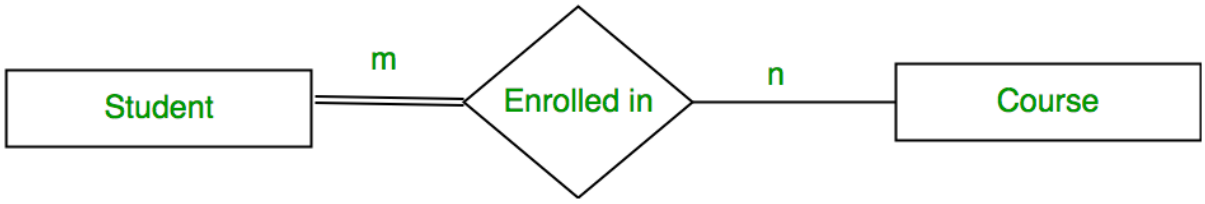
Participation Constraint

Participation Constraint is applied to the entity participating in the relationship set.

1. Total Participation – Each entity in the entity set must participate in the relationship. If each student must enroll in a course, the participation of students will be total. Total participation is shown by a double line in the ER diagram.

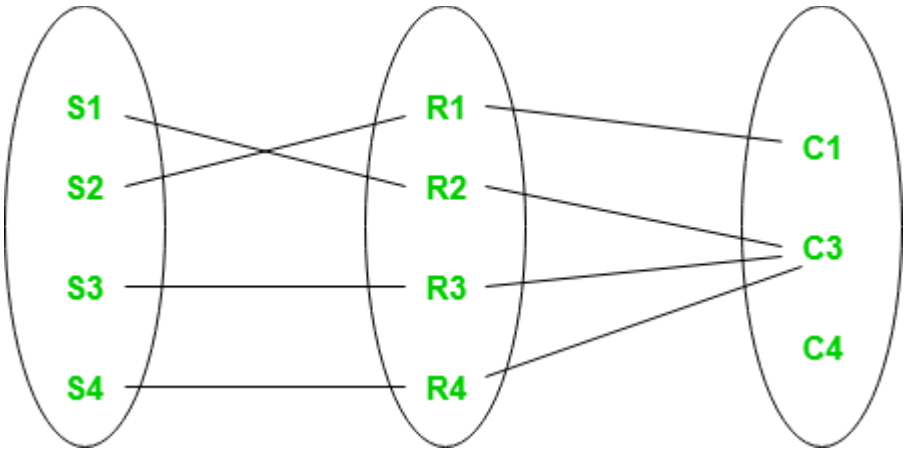
2. Partial Participation – The entity in the entity set may or may NOT participate in the relationship. If some courses are not enrolled by any of the students, the participation in the course will be partial.

The diagram depicts the 'Enrolled in' relationship set with Student Entity set having total participation and Course Entity set having partial participation.



Total Participation and Partial Participation

Using Set, it can be represented as,



Every student in the Student Entity set participates in a relationship but there exists a course C4 that is not taking part in the relationship.

Enhanced ER Model(EER Modeling)

Today the complexity of the data is increasing so it becomes more and more difficult to use the traditional ER model for database modeling. To reduce this complexity of modeling we have to make improvements or enhancements to the existing ER model to make it able to handle the complex application in a better way.

Enhanced entity-relationship diagrams are advanced database diagrams very similar to regular ER diagrams which represent the requirements and complexities of complex databases.

It is a diagrammatic technique for displaying the Sub Class and Super Class; Specialization and Generalization; Union or Category; Aggregation etc.

n addition to ER model concepts EE-R includes –

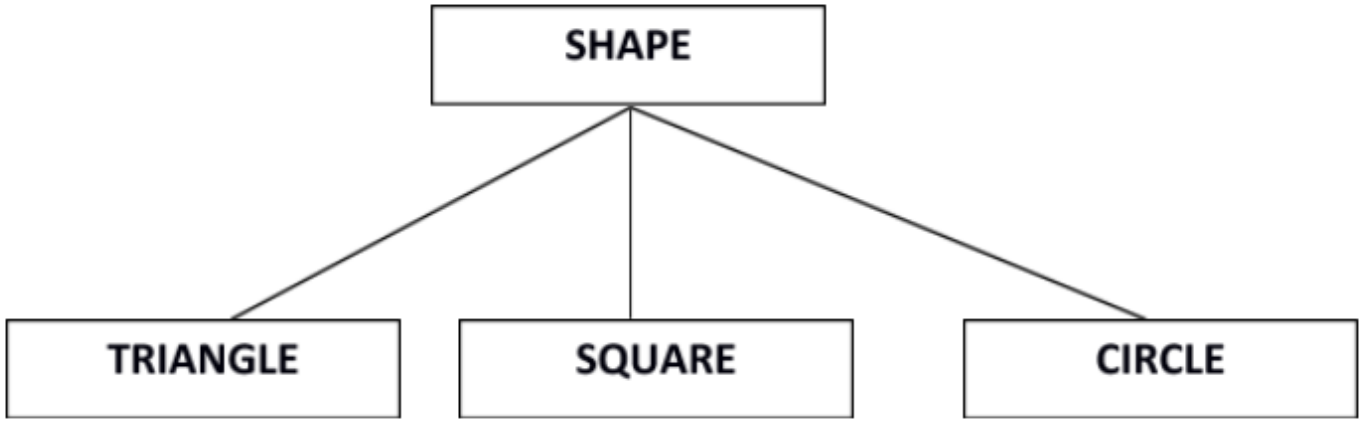
- Subclasses and Super classes.
- Specialization and Generalization.
- Category or union type.
- Aggregation.

These concepts are used to create EE-R diagrams.

Subclasses and Super class

Super class is an entity that can be divided into further subtype.

For **example** – consider Shape super class.

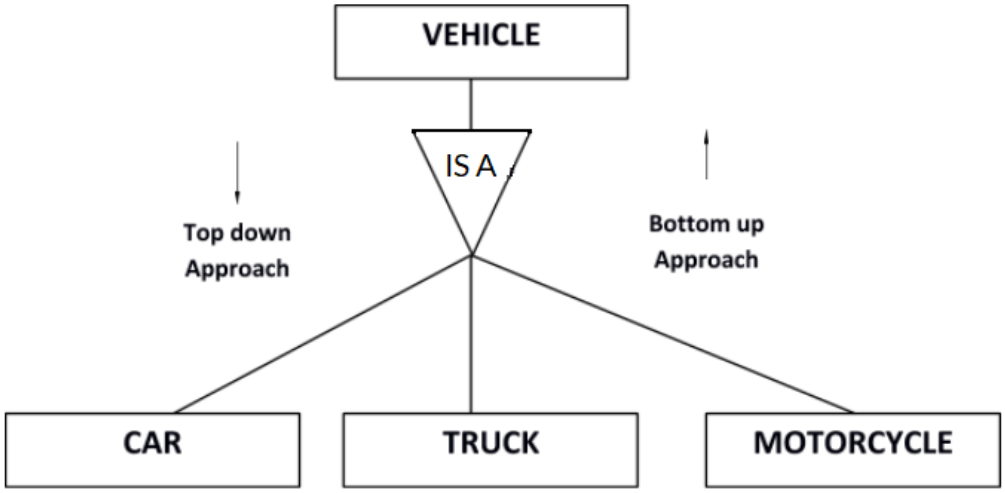


Super class shape has sub groups: Triangle, Square and Circle.

Sub classes are the group of entities with some unique attributes. Sub class inherits the properties and attributes from super class.

Specialization and Generalization

Generalization is a process of generalizing an entity which contains generalized attributes or properties of generalized entities.



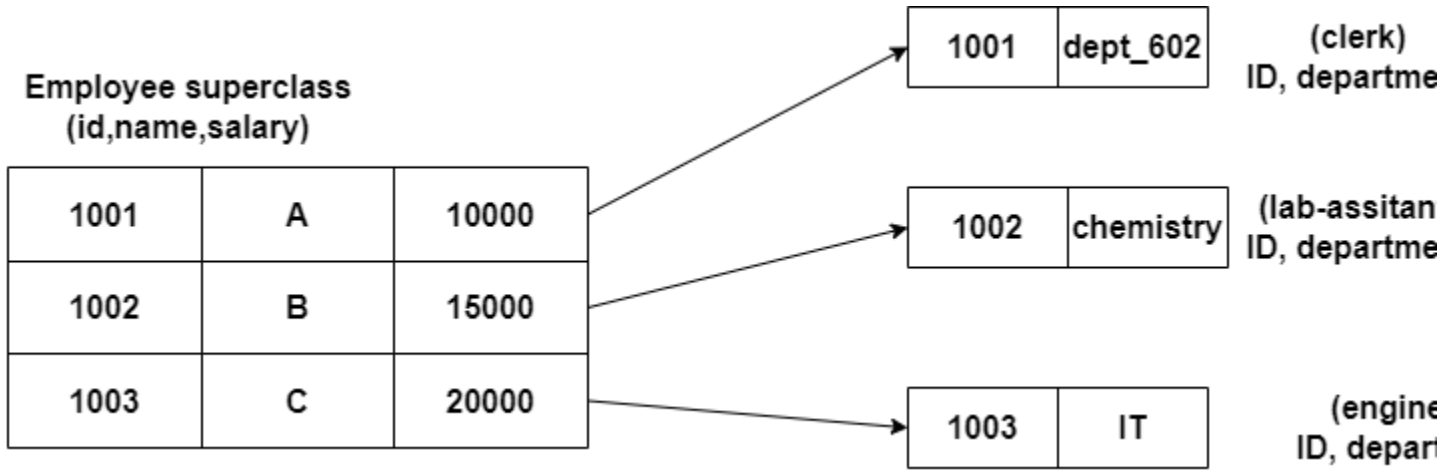
It is a Bottom up process i.e. consider we have 3 sub entities Car, Truck and Motorcycle. Now these three entities can be generalized into one super class named as Vehicle.

Specialization is a process of identifying subsets of an entity that share some different characteristic. It is a top down approach in which one entity is broken down into low level entity.

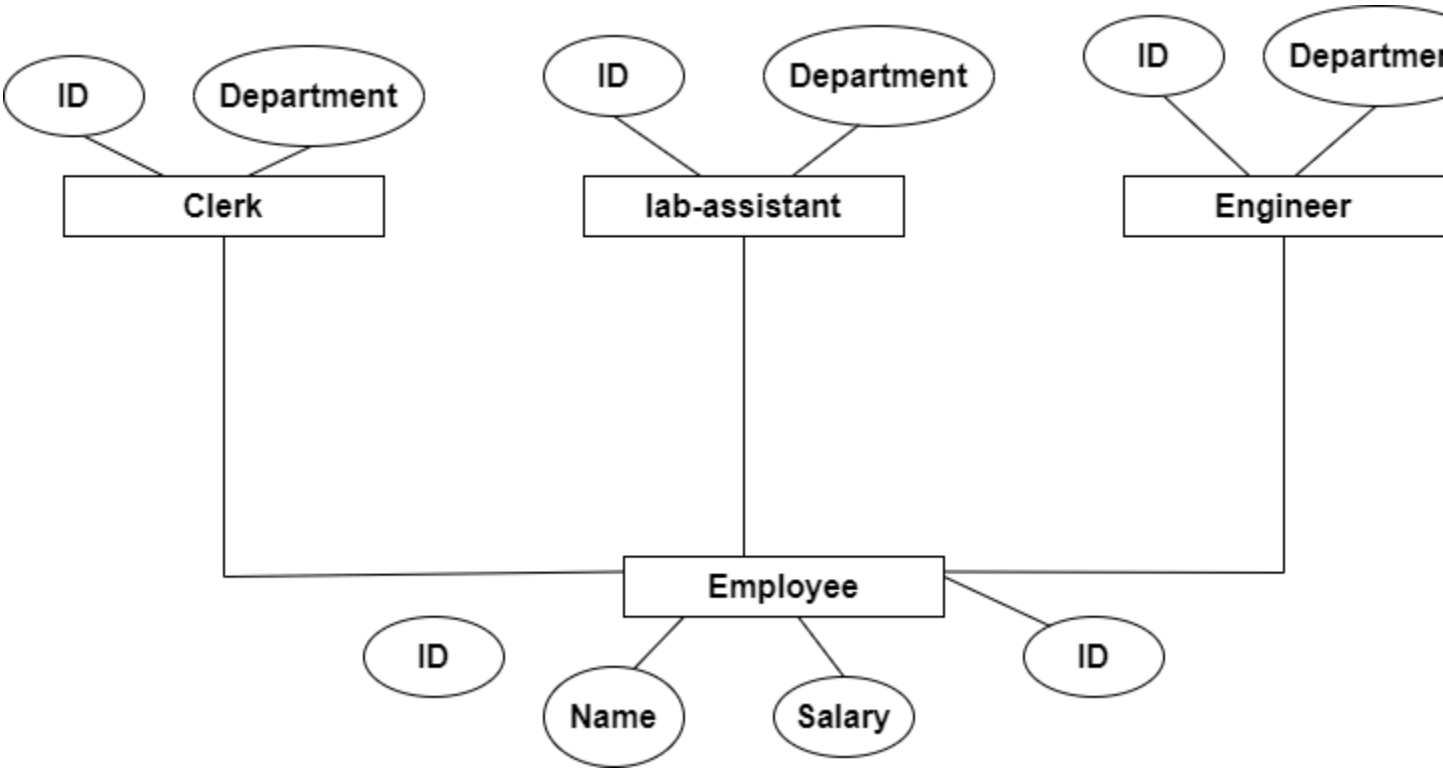
These are two normal kinds of relationships that were added to the normal ER model for enhancement. These are inspired by the object-oriented paradigm, where we divide the code into classes and objects, and in the same way, we have divided entities into subclass and superclasses. Specialized classes are called subclasses, and generalized classes are called superclasses or base classes. We can learn the concept of subclass by 'IS-A' analysis. For example, 'Laptop IS-A computer.' Or 'Clerk IS-A employee.'

In this relationship, one entity is a subclass or superclass of another entity. For example, in a university, a faculty member or clerk is a specialized class of employees. So an employee is a generalized class, and all others are its subclass.

We can draw the ER diagram for these relationships. Let's suppose we have a superclass Employee and subclasses as a clerk, engineer, and lab assistant.



The Enhanced ER diagram of the above example will look like this:



In the above example, we have one superclass and three subclasses. Each subclass inherits all the attributes from its superclass so that a lab assistant will have all its attributes, like its name, salary, etc.

Constraints

There are two types of constraints on subclasses which are described below:

- **Total or Partial:**

A total subclass relationship is one where the union of all the subclasses is equal to the superclass. It means if every superclass entity has some subclass entity, then it is called a total subclass relationship. Let's suppose if the union of all the subclasses (engineer, clerk, lab assistant) is equal to the total employee. Then the relationship is total. In the above example, it is a total relationship.

If all the entities of a superclass are not associated with a subclass, then it is called a partial subclass relationship.

- **Overlapped or Disjoint:**

If any entity from the superclass is associated with more than one subclass, then it is known as overlapped subclassing, and if it is associated with zero or only one subclass, then it is called disjoint subclassing.

Multiple Inheritance

When one subclass is associated with more than one superclass, then this phenomenon is known as multiple inheritance. In multiple inheritance, the attributes of the subclass will be the union of all the superclass attributes which are associated with it. For example, a teacher is a subclass that can be associated with the superclass of an employee and a superclass of faculty. In the same way, a monitor in the class can be a subclass of a student superclass as well as an alumni superclass.

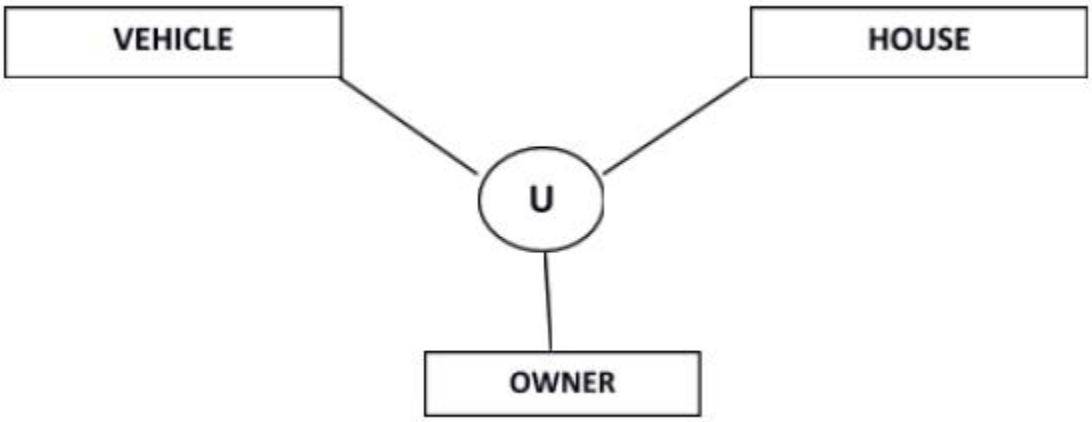
UNION

UNION is a different topic from subclassing. Let's suppose we have a vehicle superclass, and we have two subclasses, car and bike. These two subclasses will inherit the attributes from the vehicle superclass.

Now we have a UNION of those vehicles which are RTO registered, so we have a UNION of cars and bikes, but they will inherit all the attributes from the vehicle superclass.

Union

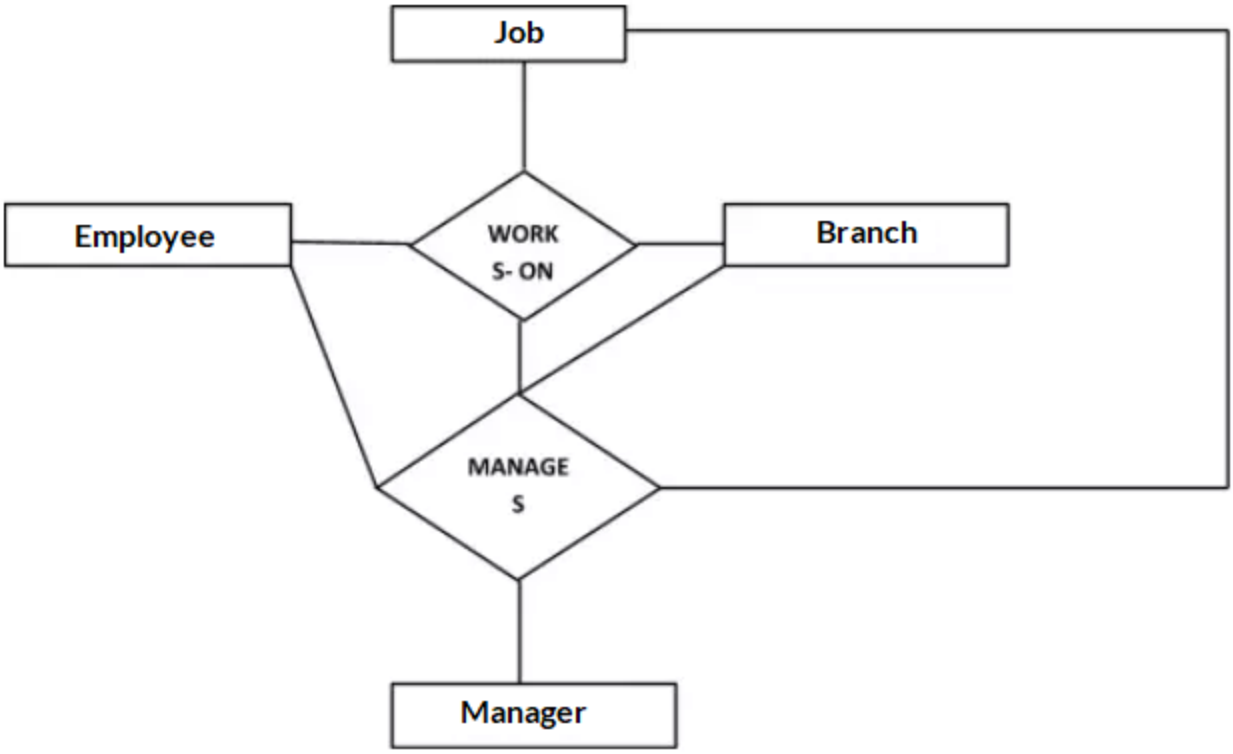
Relationship of one super or sub class with more than one super class.



Owner is the subset of two super class: Vehicle and House.

Aggregation

Represents relationship between a whole object and its component.



Consider a ternary relationship Works_On between Employee, Branch and Manager. Now the best way to model this situation is to use aggregation, So, the relationship-set, Works_On is a higher level entity-set. Such an entity-set is treated in the same manner as any other entity-set. We can create a binary relationship, Manager, between Works_On and Manager to represent who manages what tasks.

Complex DataModel Relationship

A data model is a visual representation of an organization's data structures, processes, and relationships. It serves as a blueprint for the organization's data management system, describing how data is stored, accessed, and updated.

A complex data model can be challenging to comprehend because it contains many interrelated data elements, each with its attributes and relationships. It may also include data hierarchies, data aggregations, and data transformations, making it difficult to navigate and extract meaningful insights.

To understand a complex data model, it is essential to break it down into its fundamental components and analyze each part individually. This may involve reviewing data dictionaries, entity-relationship diagrams, and other data modeling documentation.

Additionally, it can be helpful to speak with subject matter experts and data stakeholders to gain a deeper understanding of the data and its implications for the organization. By fully understanding the data model and how it relates to the organization's goals and processes, it is easier to identify areas for simplification and improvement.

Importance of Simplifying Data Models

- Data models are essential to organize complex data systems. However, as data systems grow, data models tend to become more complicated. Simplifying data models is vital since it helps to improve efficiency, speed, and accuracy of data processing. A complex data model may have redundant fields, duplicate data records, and obscure relationships, which may lead to confusion, errors, and inconsistencies. Simplifying data models enables users to focus on core data elements, identify meaningful relationships, and access necessary information quickly. It also makes the maintenance, testing, and reporting of data more straightforward.
- Simplification of data models requires a systematic approach, including identifying key data elements, organizing data into categories or classes, defining relationships between categories, and eliminating redundant data. In some cases, it may require eliminating data duplication, improving naming conventions, and simplifying data hierarchies. The use of data modeling software, entity-relationship diagrams, and automated data mapping tools can help simplify data models.
- It is essential to involve stakeholders in the data modeling process to ensure that the data model reflects the business needs of the organization. Regularly reviewing and updating the data model and documenting changes are critical practices to maintain the integrity of the data model. Testing data model changes before implementation can prevent errors and inconsistencies.
- In summary, simplifying data models is a critical process in data management that enables organizations to improve efficiency, accuracy, and speed in data processing. The use of best practices, tools, involvement of stakeholders, and regular review and updating are all essential steps to simplify data models effectively.

Steps to Simplify Complex Data Models

Identify the Key Data Elements

- Identifying the key data elements is the first step to simplifying complex data models. This process involves pinpointing the most relevant pieces of information that are necessary for the business or project. These key data elements should accurately represent the main objectives of the data model, which could be anything from customer demographics to supply chain information.
- To identify the key data elements, it's important to understand the context in which the data model will be used and what information is required to achieve specific outcomes. This could involve discussions with stakeholders or analyzing existing relevant data sources.
- It's important to consider factors such as relevance, accuracy, and completeness when determining which data elements are key. It's not necessary to include every piece of data in the model, just the information that is truly necessary to achieve the desired results.
- By identifying the key data elements, the data model can be kept simple and focused, which will make it easier to maintain and update in the future. A clear and concise data model will also be easier for end-users to understand and navigate, leading to better decision-making and improved business outcomes.

Organize Data Elements into Categories

Organizing data elements into categories is an effective technique to simplify complex data models. Here's how to do it:

- Sort the data elements into groups that share common characteristics or attributes.
- Label the categories in a meaningful and descriptive way.
- Remove any data elements that do not fit into any category.
- Consider the relationships between categories and rearrange them as necessary.
- Use subcategories to further group related data elements.
- Keep the number of categories to a minimum for ease of understanding.
- Make sure the categories accurately represent the scope and purpose of the data model.
- Apply consistent rules and standards for categorizing data elements across different models and projects.

By organizing data elements into categories, you can create a clear and logical structure for your data model. This makes it easier to understand and use by stakeholders and helps to avoid errors and redundancies.

Define Relationships between Categories

To simplify complex data models, it is essential to define relationships between categories. Here are the key points to consider when defining relationships between categories:

- Determine the types of relationships: Identify whether the relationships between categories are one-to-one, one-to-many, or many-to-many.

- Describe the relationships: Use clear and concise language to describe the relationships between categories. This will help to ensure that all stakeholders understand the relationships and their implications.
- Document the relationships: Once the relationships have been defined, document them in an easily accessible location. This will help to ensure that the relationships are clear and accessible to all stakeholders.
- Visualize the relationships: Use diagrams or other visual aids to help stakeholders understand the relationships between categories. This can help to make the relationships more tangible and easier to understand.
- Refine the relationships: As the data model evolves, it may be necessary to refine the relationships between categories.

Be sure to regularly review and update the relationships to ensure that they accurately reflect the data model.

Eliminate Redundant Data Elements

- Eliminating redundant data elements is an important step in simplifying complex data models. Redundant data elements refer to those that are repeated or duplicated in different parts of the data model, which can cause confusion and make the data model more difficult to understand.
- To eliminate redundant data elements, it is necessary to carefully examine the data model and identify any elements that are repeated unnecessarily. These duplicate elements can then be removed or consolidated into a single entity, streamlining the data model and making it easier to use and maintain.
- When eliminating redundant data elements, it is important to strike a balance between simplicity and completeness. While it may be tempting to remove as many duplicate elements as possible, it is essential to ensure that all necessary data elements are included in the model to avoid data loss or inaccuracies.
- Eliminating redundant data elements can also lead to increased efficiency in data processing and analysis, as there are fewer elements to process and fewer chances for errors to occur. By following this step, users can significantly improve the accuracy, speed, and usefulness of the data model.

Simplify Data Hierarchies

- Data hierarchies refer to how data elements are grouped and structured in a model.
- To simplify data hierarchies, start by identifying the key data elements and organizing them into categories or groups.
- Eliminate any redundant categories or groups that do not add value to the model.
- Refine the remaining categories and groups to have clear and distinct relationships between them.
- Avoid creating too many levels in the hierarchy, as this can lead to confusion and complexity.
- The goal of simplifying data hierarchies is to improve the clarity and usability of the data model for all users.

Tools to Simplify Complex Data Models
Data Modeling Software

- Data Modeling Software is a type of software that assists in creating, updating and managing data models. These software tools allow users to visually design different aspects of a database, including tables, relationships, attributes, and constraints. The goal of data modeling software is to simplify the process of creating and maintaining complex data models.
- These software tools come with pre-built templates for commonly used data modeling frameworks, including Entity Relationship Diagrams and Unified Modeling Language (UML) diagrams. This not only speeds up the process of creating a data model but also ensures consistency across the organization.
- Some data modeling software also provides features like data mapping, reverse engineering, and version control. Data mapping, for example, can help map data between two different systems, while reverse engineering allows users to create a visual representation of the existing database.
- Most data modeling software packages also contain collaboration features, allowing multiple users to work on the same model simultaneously. This can lead to increased efficiency, as the team can work together on the model from different locations.
- Overall, data modeling software is an essential tool for organizations looking to simplify the process of creating and managing data models. By providing a visual representation of data flow, it ensures that all stakeholders can understand the data assets of the organization, reducing the risk of data inconsistencies and ultimately contributing to better business outcomes.

Automated Data Mapping

- Automated Data Mapping is the process of using software tools to automatically analyze complex data models and create simplified representations of the relationships between data elements. This technology can be particularly useful for large and complex data sets, where manual data mapping can be time-consuming and error-prone.
- Automated Data Mapping software works by first analyzing the structure of the existing data model, identifying key data elements and their relationships to each other. The software then creates a simplified model, highlighting the most important data relationships and eliminating redundant data elements.
- One of the biggest advantages of Automated Data Mapping is that it can be implemented quickly and at scale, making it an ideal solution for organizations with large data sets to manage. Additionally, this technology can be used to analyze data in real-time, making it easier to identify and respond to changes in the data model.
- However, it is important to note that Automated Data Mapping should not be seen as a complete replacement for manual data mapping and analysis. While the software can do much of the heavy lifting in terms of simplifying the data model, human analysis is still necessary to ensure that the resulting model accurately reflects the needs of the organization and its stakeholders.

Entity-Relationship Diagrams

An Entity-Relationship Diagram (ERD) is a graphical representation of the entities and their relationships to each other in a database system. It is used to define the data schema, which includes the organization and structure of the data, as well as the relationships and constraints that exist between data elements.

The entities in an ERD represent the objects or concepts that exist in the system being modeled, such as customer, product, order, etc. The relationships between these entities capture the associations and dependencies that exist between them, such as a customer placing an order or a product being part of an order.

ERDs use three basic elements to represent the entities, relationships, and attributes of a database system. These elements are:

- **Entities:** Objects or concepts in the system being modeled, represented by rectangles.
- **Relationships:** Associations or dependencies between entities, represented by diamonds.
- **Attributes:** Characteristics or properties of an entity, represented by ellipses.

ERDs are useful in simplifying complex data models by visualizing the relationships between entities. They can also help in identifying redundant data and improving the efficiency of the data storage. ERDs are widely used in the software development industry and are an essential tool for designing a database schema that meets the needs of the organization.

Overall, ERDs provide a clear and concise overview of the data schema in a database system. They are an invaluable asset for database designers, developers, and stakeholders in the data modeling process.

Best Practices for Simplifying Complex Data Models

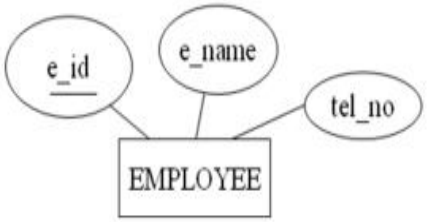
Regularly Review and Update the Data Model
To ensure the continued effectiveness of a simplified data model, it's essential to conduct regular reviews and updates. This process involves:

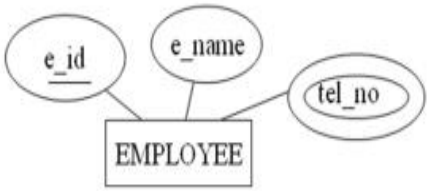
- Assessing whether the data model meets current and future business needs.
- Identifying any new data elements or categories that need to be added.
- Removing any outdated or irrelevant elements.
- Checking that relationships between categories still make sense.
- Testing that changes to the data model don't break existing systems or processes.
- Updating documentation to reflect any changes.
- Communicating any changes to stakeholders who may be affected by them.
- Encouraging feedback on the changes made to continually improve the data model.

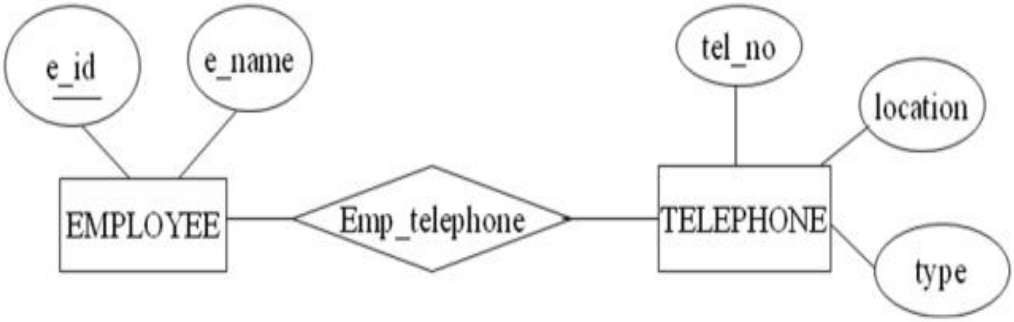
DBMS – ER Design Issues

1. Choosing Entity Set vs Attributes

The decision to utilize an entity set or attribute in a model depends on the structure of the actual business and the meaning connected with its attributes. However, it can result in an error if the user attempts to use an primary key of an Entity Set as an attribute for a different entity set. In such cases, it is recommended to use the relationship instead. Additionally, while the primary key attributes are implicit in the relationship set , it is still designated in the relationship sets .

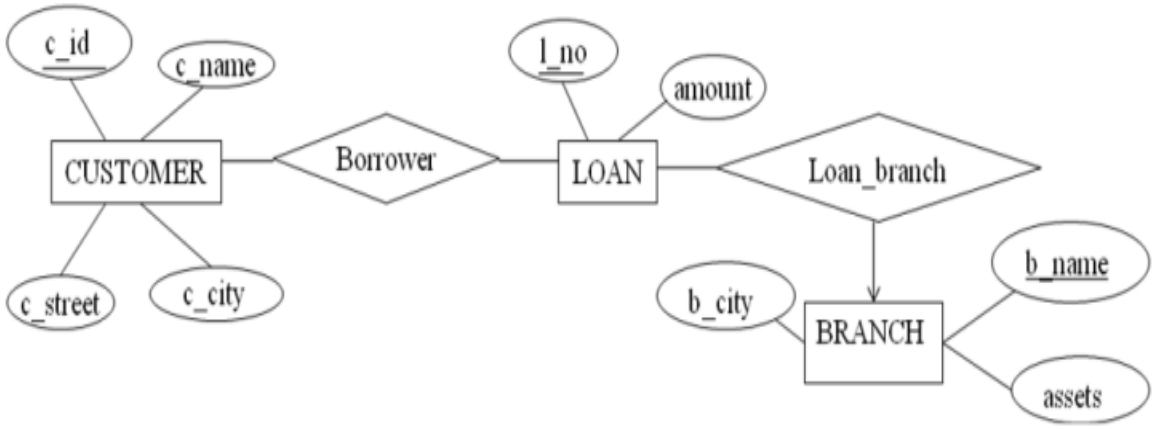


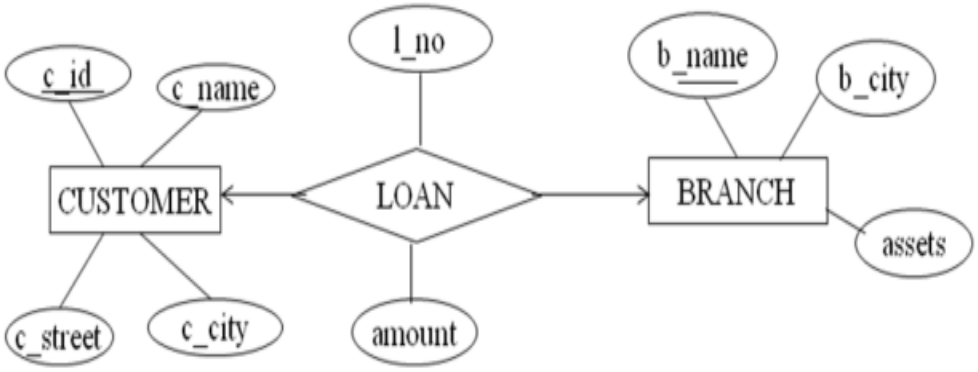




2. Choosing Entity Set vs. Relationship Sets

It is difficult to examine if an object can be best expressed by an entity set or relationship set. To understand and determine the right use, the user need to designate a relationship set for describing an action that occurs in-between the entities. If there is a requirement of representing the object as a relationship set, then its better not to mix it with the entity set.



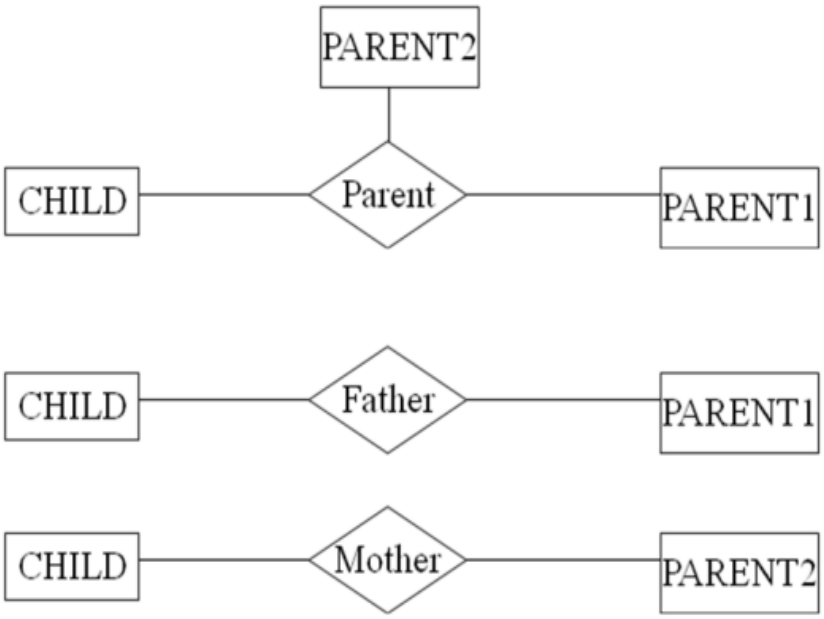


3. Choosing Binary vs n-ary Relationship Sets

The relationships described in an **ER diagrams** are binary. The **n-ary** relationships are those where entity sets are more than two, if the entity sets are only two, their relationship can be termed as binary relationship.

The n-ary relationships can make ER design complex, however the good news is that we can convert and represent any n-ary relationship using multiple binary relationships.

For example, we can create and represent a ternary relation ship 'parent' that may relate to a child, his father, as well as his mother. Such relationship can also be represented by two binary relationships i.e, mother and father, that may relate to their child. Thus, it is possible to represent a non-binary relationship by a set of distinct binary relationships.



4. Placing Relationship Attributes

The cardinality ratio of a relationship can affect the placement of relationship attributes:

• One-to-Many: Attributes of 1:M relationship set can be repositioned to only the entity set on the many side of the relationship

• One-to-One: The relationship attribute can be associated with either one of the participating entities

• Many-to-Many: Here, the relationship attributes can not be represented to the entity sets; rather they will be represented by the entity set to be created for the relationship set

Disadvantages/Limitations of EER Diagrams

- The EER diagrams have many constraints and come up with limited features.
- Faults in the scoring of data can happen, plus also there could be an error in the application.
- Calculated on past data and therefore, cannot predict the future.
- Having the attributes displayed on the page (as symbols) can make it harder to read (ORM is worse for this because of having relationships between each of the attributes). Although an EERD will not tend to consolidate these attributes in the same way an ORM can...
- In general, there are so many different variations of the ER approach (a bit like linux), one of the main problems is that there is no set standard. You will see some using 'crows feet' to symbolise a Many relationship, others wont etc.
- It is not as accurate as the ORM approach meaning you would need to work harder to capture all the business rules (go around asking more questions to get an idea how their system should be transformed into an accurate data model)